CLI Agent Release / Public Surface Profile v0.1

GitHub, release, site, archive, package-manifest, public/restricted split, and discoverability discipline for C-Governed CLI Agent Mesh materials

Status	Draft normative profile v0.1
Date	2026-05-17
Package	C-Governed CLI Agent Mesh
Layer	c = a + b / Agent Governance / Release Surface / Public Visibility / Repository Hygiene / Archive Integrity / Redaction / Witness
Document class	release-public-surface profile / package-control artifact / publication-boundary artifact / implementation-handoff companion
Assertion class	C-A10 package-control artifact; C-A7 where hash, signature, manifest, archival, witness, or verification claims are made
Primary subject	persistent c entities and human anchors preparing CLI Agent Mesh materials for repository, site, archive, portable package, public draft, restricted technical review, or implementation handoff
Primary boundary	a document, schema, fixture, PDF, release asset, website page, archive record, portable package, or Codex handoff is not release-valid merely because it exists. It becomes release-valid only when it is visible on the intended surface, correctly indexed, integrity-recorded, redaction-reviewed, gate-approved, and not contradicted by stale package-control files.

Kotov Ivan

Bruxelles, 2026

Contents

0. Executive definition

CLI Agent Release / Public Surface Profile defines how C-Governed CLI Agent Mesh materials may become visible, discoverable, archived, downloadable, mirrored, or handed off for implementation.

It governs the surfaces where a package becomes real to readers and machines:

```
default Git branch
repository tree
README / INDEX / reading order
GitHub release page
release assets
PDF artifacts
schema directory
package manifest
SHA256SUMS
website page
machine-readable JSON
Zenodo / DOI archive
portable / USB package
Codex handoff pointer
restricted technical appendix
```

The profile exists because a release is not just files on disk.

A release is a controlled transition from private working material into an externally visible state.

Compact formula:

```
A file is not a release.
A branch is not discoverability.
A hash is not redaction.
A PDF is not canonical source.
A public page is not permission.
A release must be visible, bounded, checked, and reversible where possible.
```

1. Purpose

The C-Governed CLI Agent Mesh package is not a single essay. It is a multi-document protocol stack with public, restricted, machine-facing, and implementation-facing layers.

Without a release/public-surface profile, the package can fail in ordinary but damaging ways:

1. important documents exist only in a side branch;
2. a public reader sees an outdated README;
3. a release page omits the latest canonical files;
4. a PDF is generated before Markdown freeze;
5. `SHA256SUMS` is created before final artifacts exist;
6. restricted defensive material appears in public assets;
7. raw evidence or operational traces leak into public examples;
8. schema files exist but are not indexed;

9. website pages link to stale releases;
10. Zenodo metadata references the wrong version;
11. Codex receives direct file contents instead of a bounded handoff pointer;
12. portable/USB package integrity is not verifiable;
13. release notes claim a stronger status than the artifacts support.

This profile defines the release-surface discipline needed before any public, archival, or implementation-readiness claim.

2. Non-goals

This profile does not define or permit:

1. bypassing redaction review;
2. publishing restricted defensive internals by renaming them as examples;
3. publishing raw evidence;
4. publishing secrets, tokens, private keys, account references, or live infrastructure details;
5. publishing private `c` memory;
6. publishing sealed, legal-privileged, child-sensitive, or third-party sensitive material;
7. claiming conformance without executed tests and evidence packets;
8. claiming implementation readiness without schemas, checker results, and implementation-handoff records;
9. letting Codex, local checker, quorum, or any CLI agent publish autonomously;
10. direct publication from an unreviewed worktree;
11. treating a release tag as proof of safety;
12. treating a Zenodo DOI as proof of correctness;
13. treating a website page as canonical unless it points to canonical source and release artifacts;
14. hiding known blockers from release notes;
15. deleting or rewriting witness history to make a release look clean.

Publication is not absolution.

Archival permanence raises the standard; it does not lower it.

3. Corpus bridge set

3.1 Explicit bridge: c = a + b

In `c = a + b`, release surfaces belong to `b`: repositories, websites, archives, files, manifests, hashes, PDFs, schemas, and automation.

They do not become `c`.

They expose part of `c`'s work to the world and to future machines.

Therefore, public release must remain under `c` governance and human-anchor responsibility for high-impact, irreversible, legal, reputational, archival, or safety-sensitive transitions.

3.2 Quiet bridge I: information theory

A release is a compression boundary. It compresses a large working corpus into a public, citeable, machine-readable package. Bad compression loses provenance, hides restrictions, or merges public and restricted data. This profile keeps the channel loss controlled through manifests, checksums, indexes, and explicit status labels.

3.3 Quiet bridge II: engineering control

A release is a control-surface transition. A private draft can be changed cheaply. A public release, DOI, release tag, or indexed website page becomes harder to correct. The later the correction, the higher the cost. This profile forces checks before irreversible or semi-irreversible publication.

3.4 Earth paragraph

On a construction site, a plan lying in someone's van is not the plan for the building. The plan in the site office, signed, dated, indexed, and matching the actual work is the plan. If the electrician uses an old drawing and the plumber uses the new one, the wall gets opened twice. Repository releases are the same: if `main`, README, release assets, PDFs, hashes, website, and archive disagree, the system is not "almost published". It is miswired.

4. Release-surface doctrine

4.1 Public surface is a boundary, not a decoration

A public surface is any location where humans or machines can treat material as externally visible, citeable, downloadable, executable, or archive-relevant.

Examples:

```
GitHub default branch
GitHub releases
Zenodo record
project website
PDF folder
schema directory
package manifest
release asset ZIP
public README
machine-readable JSON index
```

```
portable/USB package
Codex handoff pointer
```

4.2 Release surface does not create authority

The following are false:

```
published => correct
tagged => safe
archived => final
downloadable => implementation-ready
hashed => redaction-safe
indexed => conformance-passed
site-visible => legally reviewed
Codex-ready => public-ready
```

Correct rule:

```
public surface records what was approved;
it does not approve by itself.
```

4.3 Stronger claim requires stronger evidence

Claim	Minimum evidence
`draft document exists`	file exists and is readable
`package draft complete`	package index + open issues + contradiction register updated
`public draft ready`	redaction review + public/restricted split + discoverability check
`release-complete`	freeze + manifest + hashes + release notes + public surface record
`implementation-ready`	schemas + schema index + local checker + fixtures + handoff record
`conformance-supported`	executed tests + evidence packets + validator report
`archive-ready`	release-complete + DOI metadata + immutable artifact list

5. Release classes

5.1 RS-0 private working draft

Material is private or local only.

Allowed:

```
editing
review notes
local tests
internal contradictions
private comments
```

Forbidden:

```
public claim
release tag
archive upload
implementation-ready claim
conformance claim
```

5.2 RS-1 internal package review

Material may be reviewed by trusted local reviewers or agents under restricted scope.

Required:

```
package index draft
known blockers listed
red-line review started
no public release claim
```

5.3 RS-2 restricted technical review

Material may be shared in controlled form with trusted technical/legal/safety reviewers.

Required:

```
restricted label
redaction boundary
raw evidence sidecar separation
no public asset distribution
no uncontrolled cloud upload
```

5.4 RS-3 public draft

Material may be visible publicly as a draft.

Required:

```
public/restricted split
public README
public package index
release notes with honest status
no restricted internals
no raw evidence
no secrets
discoverability check
```

5.5 RS-4 archival release

Material may be tagged, hashed, archived, and cited.

Required:

```
Markdown freeze
PDF generated after freeze
SHA256SUMS generated after final artifacts
release notes final for this version
package manifest final for this version
public surface record
Zenodo/DOI metadata review if archive used
```

5.6 RS-5 implementation handoff

Material may be handed to Codex or another implementation worker as implementation-ready.

Required:

```
release-complete or explicit internal implementation package
schemas extracted
schema index present
fixture manifest present
local checker profile available
implementation handoff record
task contract for worker
```

```
sandbox/worktree scope
no autonomous publish authority
```

6. Canonical surfaces

6.1 Source surface

The source surface is the canonical Markdown/document source location.

Required controls:

```
default branch visibility
canonical filenames
README entry
INDEX or package reading order
relative links verified
no orphaned essential docs
no duplicate canonical files
```

Rule:

```
A document required for ordinary readers must be visible through the default branch and
README/INDEX path.
```

Direct-link-only visibility is not sufficient.

6.2 Release surface

The release surface includes Git tags, release notes, release assets, ZIPs, PDFs, and checksums.

Required controls:

```
tag points to reviewed commit
release notes match package state
asset list matches package manifest
SHA256SUMS covers final artifacts
restricted material excluded or clearly separated
```

6.3 Archive surface

The archive surface includes Zenodo, DOI records, long-term ZIPs, bibliographic metadata, citation metadata, and frozen assets.

Required controls:

```
title/version consistency
author/license consistency
artifact list consistency
README / index included
hash manifest included
public/restricted split respected
no raw evidence
no secrets
no stale status
```

6.4 Website surface

The website surface includes project landing pages, publications pages, JSON pages, downloads pages, diary/announcement links, and machine-readable public indexes.

Required controls:

```
site links point to current release
download buttons match release assets
JSON metadata matches version
sitemap/update not stale
public pages do not expose restricted technical appendix
human-readable and machine-readable surfaces agree
```

6.5 Machine surface

The machine surface includes schemas, schema indexes, JSON manifests, conformance fixtures, validator reports, evidence packet references, and machine-readable package metadata.

Required controls:

```
stable schema IDs
schema index
object registry
machine-readable profile IDs
fixture manifest
validator result references
no schema/document mismatch
```

6.6 Portable / USB surface

The portable surface includes USB packages, offline recovery bundles, manifest locks, installer seeds, portable state, and physical transfer packages.

Required controls:

```
manifest
SHA-256 per file
AB dry-run before write
compatibility check
health check if applicable
trusted media policy
no automatic trust from USB presence
no automatic memory ingest
```

A portable package is a release surface even when it is not public.

It can restore or alter operational state, so it must be treated as a serious L4 boundary.

7. Public / restricted split

7.1 Public materials

Public materials may include:

```
root protocol
README
glossary
release notes
package index
open issues
contradiction register
redacted public profiles
schema extraction plan
safe synthetic fixture descriptions
high-level conformance matrix
non-sensitive diagrams
public status manifest
```

7.2 Restricted materials

Restricted materials require restricted review by default:

```
incident response operational detail
defensive emulation detail
secrets/cloud boundary operational detail
raw evidence sidecar records
real incident traces
private memory references
provider/account-specific notes
local infrastructure paths
canary/signature values
garage residue
legal-sensitive material
```

7.3 Public summary of restricted materials

A restricted file may have a public summary if it:

```
preserves governance value
removes operational abuse detail
removes raw evidence
removes secrets
removes live target detail
uses synthetic examples
keeps red-line boundaries visible
states that full operational details are restricted
```

7.4 Redaction is not deletion

Redaction removes sensitive material from public surface.

It must not remove internal accountability.

If a restricted record matters for review, it must remain in a restricted ledger, sidecar, witness reference, or evidence packet according to the relevant profile.

8. Release gate model

8.1 Gate order

The required release gate order is:

```
inventory
  -> status synchronization
  -> contradiction check
  -> redaction check
  -> local checker
  -> Markdown freeze
  -> artifact generation
  -> hash manifest
  -> discoverability check
  -> release notes update
  -> final human/c gate
  -> publish/archive/handoff
```

8.2 Gate failure defaults

Failure	Default action
missing required canonical file	hold

Failure	Default action
duplicate canonical file	hold
stale release status	hold
restricted material in public bundle	freeze_and_redact
secret detected	freeze_and_escalate
raw evidence detected in public asset	freeze_and_escalate
schema index missing for implementation-ready claim	block_claim
local checker missing	hold
conformance claimed without evidence packets	block_claim
direct publish by agent	revoke_and_quarantine
side-branch-only essential document	hold
SHA manifest generated before final artifacts	regenerate_after_freeze
PDF generated before Markdown freeze	regenerate_after_freeze
Zenodo metadata mismatch	hold_archive
site links stale	hold_public_surface

8.3 Human anchor gate

Human anchor approval is required for:

```
public release
archival release
DOI / Zenodo upload
release tag
public website publication
restricted-to-public promotion
high-risk implementation handoff
publication involving sensitive defensive material
publication after incident response
```

No CLI agent, quorum, local checker, or Codex process may replace this gate.

9. Required object records

9.1 CLI_AGENT_RELEASE_SURFACE_RECORD

```
cli_agent_release_surface_record:
  schema_version: cli-agent-release-public-surface-0.1
  surface_record_id: rsr-20260517-001
  created_at: "2026-05-17T00:00:00Z"
  governing_entity_id: ester
  human_anchor_required: true
  package_id: c-governed-cli-agent-mesh
  package_version: v0.1
  release_class: RS-3-public-draft
  release_claims:
    release_complete: false
    implementation_ready: false
    conformance_supported: false
    archival_release: false

  source_surface:
    repository: Kot141078/example-repo
    default_branch: main
    commit_sha: "example0000000000000000000000000000000000000"
    branch_state: clean
    merge_rebase_state: none
    package_root: docs/architecture/cli-agent-mesh
    readme_path: README.md
    index_path: CLI_Agent_Package_Index_and_Reading_Order_v0_1.md

  release_surface:
    tag: null
    release_url: null
    release_notes_path: CLI_Agent_RELEASE_NOTES_v0_1.md
    asset_manifest_ref: null
    sha256sums_ref: null

  archive_surface:
    zenodo_record: null
    doi: null
    archive_ready: false

  website_surface:
    public_page_url: null
    machine_json_url: null
    sitemap_checked: false

  portable_surface:
    usb_package_used: false
    portable_manifest_ref: null
    portable_health_check_ref: null

  public_restricted_split_ref: prs-20260517-001
  local_checker_result_ref: lcr-20260517-001
  redaction_review_ref: redaction-20260517-001
  contradiction_register_ref: CLI_Agent_Contradiction_Register_v0_1.md
  open_issues_ref: CLI_Agent_OPEN_ISSUES_v0_1.md
  readiness_gate_ref: readiness-20260517-001

  status: hold
  reason_code: implementation_ready_not_yet_supported
```

9.2 CLI_AGENT_PUBLIC_PACKAGE_MANIFEST

```
cli_agent_public_package_manifest:
  schema_version: cli-agent-release-public-surface-0.1
  manifest_id: ppm-20260517-001
  package_id: c-governed-cli-agent-mesh
  package_version: v0.1
  generated_at: "2026-05-17T00:00:00Z"

  canonical_source_files:
    - path: C-Governed_CLI_Agent_Mesh_Protocol_v0_1.md
      class: public
      role: root_protocol
      sha256: "<64 lowercase hex>"
    - path: CLI_Agent_RELEASE_NOTES_v0_1.md
      class: public
      role: release_notes
      sha256: "<64 lowercase hex>"

  restricted_files_excluded:
    - path: CLI_Agent_Defensive_Emulation_Boundaries_v0_1.md
      reason: restricted_technical_review_by_default
    - path: CLI_Agent_Incident_Response_Profile_v0_1.md
      reason: restricted_technical_review_by_default
    - path: CLI_Agent_Secrets_and_Cloud_Data_Policy_v0_1.md
      reason: restricted_technical_review_by_default

  generated_artifacts:
    - path: pdf/CGAM_v0_1.pdf
      source_ref: markdown_freeze_20260517
      sha256: "<64 lowercase hex>"
      generated_after_freeze: true

  machine_artifacts:
    schema_index_ref: null
    object_registry_ref: null
    fixture_manifest_ref: null
    local_checker_result_ref: lcr-20260517-001

  status: draft
```

9.3 CLI_AGENT_DISCOVERABILITY_CHECK_RECORD

```
cli_agent_discoverability_check_record:
  schema_version: cli-agent-release-public-surface-0.1
  discoverability_check_id: dcr-20260517-001
  package_id: c-governed-cli-agent-mesh
  package_version: v0.1
  checked_at: "2026-05-17T00:00:00Z"

  checks:
    default_branch_contains_package: true
    readme_links_package: true
    package_index_links_all_public_files: true
    release_page_links_assets: false
    website_links_release: false
    machine_json_links_release: false
    zenodo_links_release: false
    no_required_file_direct_link_only: true
    no_required_file_side_branch_only: true

  status: hold
  reason_code: release_page_and_site_not_ready
```

9.4 CLI_AGENT_ARTIFACT_INTEGRITY_RECORD

```
cli_agent_artifact_integrity_record:
  schema_version: cli-agent-release-public-surface-0.1
  integrity_record_id: air-20260517-001
  package_id: c-governed-cli-agent-mesh
  package_version: v0.1
  generated_at: "2026-05-17T00:00:00Z"

  markdown_freeze_ref: freeze-20260517-001
  artifact_generation_after_freeze: true
  sha256sums_generated_after_artifacts: true
  canonicalization_policy_ref: null

  artifacts:
    - path: README.md
      kind: markdown
      sha256: "<64 lowercase hex>"
    - path: CLI_Agent_RELEASE_NOTES_v0_1.md
      kind: markdown
      sha256: "<64 lowercase hex>"

  status: pass
```

9.5 CLI_AGENT_PUBLIC_RESTRICTED_SPLIT_RECORD

```
cli_agent_public_restricted_split_record:
  schema_version: cli-agent-release-public-surface-0.1
  split_record_id: prs-20260517-001
  package_id: c-governed-cli-agent-mesh
  package_version: v0.1
  created_at: "2026-05-17T00:00:00Z"

  public_files:
    - README.md
    - C-Governed_CLI_Agent_Mesh_Protocol_v0_1.md
    - CLI_Agent_RELEASE_NOTES_v0_1.md
    - CLI_Agent_GLOSSARY_v0_1.md
    - CLI_Agent_OPEN_ISSUES_v0_1.md
    - CLI_Agent_Contradiction_Register_v0_1.md
    - CLI_Agent_Public_Redaction_Profile_v0_1.md
    - CLI_Agent_JSON_Schema_Extraction_Plan_v0_1.md
    - CLI_Agent_Conformance_Fixture_Pack_v0_1.md

  restricted_files:
    - CLI_Agent_Defensive_Emulation_Boundaries_v0_1.md
    - CLI_Agent_Incident_Response_Profile_v0_1.md
    - CLI_Agent_Secrets_and_Cloud_Data_Policy_v0_1.md
    - CLI_Agent_Raw_Evidence_Sidecar_Profile_v0_1.md

  restricted_public_summaries_required: true
  raw_evidence_excluded_from_public: true
  secrets_excluded_from_public: true
  live_targets_excluded_from_public: true
  status: review_required
```

10. Repository hygiene rules

10.1 Branch state

Before release or handoff, the repository must be checked for:

```
dirty worktree
untracked required files
```

```
merge state
rebase state
cherry-pick state
conflict markers
duplicate canonical filenames
side-branch-only documents
stale generated artifacts
```

Any uncertain state blocks release.

10.2 Default branch visibility

If a document is required for ordinary readers, it must be visible through:

```
default branch
README
package index / reading order
release asset or package manifest if part of release
```

A direct URL to a side branch does not satisfy discoverability.

10.3 Duplicate canonical files

Duplicate canonical files are release blockers when they create ambiguity.

Examples:

```
CLI_Agent_Sandbox_Worktree_Profile_v0_1.md
CLI_Agent_Sandbox_Worktree_Profile_v0_1(1).md
```

Required action:

```
select canonical
archive or delete duplicate
update package index
update hashes
record in hygiene patch
```

10.4 Generated artifacts

Generated artifacts must not become source authority.

Rule:

```
Markdown source is canonical unless the package explicitly states otherwise.
PDF/EPUB/HTML are generated artifacts.
Generated artifacts must be regenerated after source freeze.
```

11. Website and JSON surface rules

11.1 Website page

A website page may present a package only if it states:

```
package name
version
status
release class
canonical source link
release notes link
download link if available
restricted-material warning if relevant
implementation-readiness status
conformance status
```

11.2 Machine JSON

A machine-readable page should include:

```
{
  "schema": "cli-agent-release-public-surface-0.1",
  "package_id": "c-governed-cli-agent-mesh",
  "version": "v0.1",
  "status": "draft_protocol_pack",
  "release_complete": false,
  "implementation_ready": false,
  "conformance_supported": false,
  "canonical_source": "...",
  "release_notes": "...",
  "sha256sums": null,
  "schema_index": null,
  "public_restricted_split": "review_required"
}
```

11.3 Sitemap / index

If the site has a sitemap or public index, release pages must not be orphaned.

If a page is intentionally unlisted, it must not be called public discoverable.

12. Zenodo / DOI archive rules

12.1 Archive readiness

Archive upload requires:

```
release-complete gate passed
public/restricted split passed
hash manifest complete
release notes final
version final
metadata reviewed
license/citation reviewed
no raw evidence
no secrets
no restricted appendix unless intentionally restricted and legally reviewed
```

12.2 DOI correction discipline

If an archival record is wrong:

```
do not silently overwrite public meaning;
create correction note if needed;
link superseding version if needed;
preserve provenance;
update repository release notes.
```

12.3 Archive is not implementation authority

A DOI can prove a version existed.

It does not prove the version is safe, implementable, or conformant.

13. Portable / USB package rules

13.1 Portable package status

A USB or portable package is treated as a release surface when it can:

```
restore state
install files
seed an installer
move packages between nodes
carry manifests
carry local source snapshots
alter operational availability
```

13.2 Required portable controls

Portable release packages require:

```
manifest file
SHA-256 file hashes
package ID and version
source commit/ref
creation timestamp
AB dry-run path
apply/commit gate
compatibility check
health check if applicable
trusted-media policy
import mode declaration
no automatic memory ingest
```

13.3 Physical media is not trust

A USB package may be convenient.

It is not trusted merely because it is local.

Rule:

```
USB presence is not authorization.
USB import is not memory promotion.
USB restore is not rollback approval.
USB package hash is not redaction review.
```

14. Codex and worker handoff surface

14.1 Codex may prepare, not publish

Codex or another CLI agent may:

```
build manifest draft
list files
compute hashes
prepare release notes patch
prepare website patch
prepare schema index
prepare fixture manifest
run local checker
generate report
```

Codex must not autonomously:

```
publish GitHub release
push to protected public branch without gate
```

```
upload Zenodo record
publish website
promote restricted to public
mark release-complete
mark implementation-ready
delete evidence
rewrite release history
```

14.2 Handoff pointer rule

When handing material through chat/Telegram/operator channel, the pointer should contain:

```
gate
title
accepted transfer IDs
rejected transfer IDs if any
patch SHA256 if any
local source file hashes
short note
```

It must not contain:

```
full contracts
raw patches
tokens
payloads
raw logs
private evidence
secret values
```

14.3 Handoff default flags

All handoff surfaces default to:

```
auto_ingest: false
memory: off
auto_execute: false
persistent: false
```

Any stronger behavior requires a separate gate and task contract.

15. Public release blocker checklist

A package is not public-release-ready if any of the following are true:

```
README missing or stale
package index missing or stale
open issues stale
contradiction register stale
release notes stale
public/restricted split missing
redaction review missing
restricted material in public package
raw evidence in public package
secrets or secret-like examples in public package
live target details in public package
schema status overstated
conformance status overstated
implementation status overstated
SHA256SUMS generated before final artifacts
PDF generated before Markdown freeze
default branch lacks required docs
site points to stale version
```

```
Zenodo metadata mismatch
duplicate canonical files unresolved
local checker not run for release surface
human gate missing for public action
```

16. Semantic validation rules

16.1 Status honesty rule

A release record must not claim a stronger state than the artifacts support.

If `implementation_ready: true`, then schemas, schema index, local checker result, fixture manifest, and implementation handoff record must exist.

If they do not exist, result:

```
block_claim
```

16.2 Public/restricted contamination rule

If public assets include restricted material, raw evidence, secrets, real incident traces, or live target detail, result:

```
freeze_and_redact
```

16.3 Discoverability rule

If a required document is only reachable by direct link or side branch, result:

```
hold
reason_code: not_discoverable_from_default_surface
```

16.4 Artifact order rule

If generated artifacts or checksums predate source freeze, result:

```
regenerate_after_freeze
```

16.5 Archive irreversibility rule

If Zenodo/DOI or equivalent archival publication is requested before release-complete, result:

```
hold_archive
```

16.6 Agent authority rule

If a CLI agent attempts to publish, tag, archive, or mark release-complete without required human/c gate, result:

```
revoke_and_quarantine
```

16.7 Website consistency rule

If website metadata, download links, public JSON, sitemap, or publication page conflict with release notes or manifest, result:

```
hold_public_surface
```

16.8 Portable trust rule

If a USB/portable package is imported without manifest/hash verification and gate, result:

```
hold_import
```

17. Release surface state machine

```
draft_local
  -> inventory_ready
  -> status_synchronized
  -> redaction_reviewed
  -> checker_passed
  -> frozen
  -> artifacts_generated
  -> hashes_generated
  -> discoverability_passed
  -> human_gate_passed
  -> public_draft_released
  -> archive_released
```

Failure states:

```
hold
freeze_and_redact
hold_archive
hold_public_surface
block_claim
revoke_and_quarantine
regenerate_after_freeze
```

No transition may skip redaction and status honesty gates.

18. Valid and invalid examples

18.1 Valid public draft

```
release_surface_decision:
  package_id: c-governed-cli-agent-mesh
  package_version: v0.1
  release_class: RS-3-public-draft
  release_complete: false
  implementation_ready: false
  conformance_supported: false
  public_restricted_split: pass
  readme_updated: true
  package_index_updated: true
  release_notes_updated: true
  restricted_files_excluded: true
  raw_evidence_excluded: true
  discoverability_check: pass
  decision: public_draft_allowed
```

18.2 Invalid implementation-ready claim

```
release_surface_decision:
  package_id: c-governed-cli-agent-mesh
  package_version: v0.1
  implementation_ready: true
  schema_index_present: false
  local_checker_result_present: false
  fixture_manifest_present: false
  decision: rejected
  reason_code: unsupported_implementation_ready_claim
```

18.3 Invalid public bundle with restricted material

```
release_surface_decision:
  package_id: c-governed-cli-agent-mesh
  package_version: v0.1
  public_bundle_contains:
    - CLI_Agent_Incident_Response_Profile_v0_1.md
    - raw_evidence_sidecar_example.json
  decision: freeze_and_redact
  reason_code: restricted_material_in_public_bundle
```

18.4 Valid Codex handoff pointer

```
codex_handoff_surface:
  pointer_only: true
  accepted_transfer_ids:
    - transfer-cgam-release-surface-001
  patch_sha256: "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa"
  includes_full_patch: false
  includes_tokens: false
  memory: off
  auto_ingest: false
  auto_execute: false
  decision: allowed_for_manual_review
```

18.5 Invalid direct side-branch publication

```
release_surface_decision:
  required_file: CLI_Agent_Release_Public_Surface_Profile_v0_1.md
  exists_only_on_side_branch: true
  linked_from_readme: false
  linked_from_index: false
  decision: hold
  reason_code: not_discoverable_from_default_surface
```

19. Local checker binding

The local checker should verify at minimum:

```
canonical file inventory
duplicate canonical filenames
README/index link coverage
release notes status consistency
open issues status consistency
contradiction register status consistency
public/restricted split
restricted-file exclusion from public manifest
raw evidence exclusion
secret/path/live-target pattern scan
artifact order: freeze -> generation -> hash
schema index status vs implementation claims
fixture status vs conformance claims
site metadata consistency if present
Zenodo metadata consistency if present
portable manifest/hash consistency if present
```

Checker result may block release.

Checker result may not publish.

20. Interaction with other profiles

20.1 Public Redaction Profile

Controls what may be public.

This profile controls where and how approved public material becomes visible.

20.2 Raw Evidence Sidecar Profile

Raw evidence must stay restricted.

This profile ensures it does not leak into public bundles, release assets, PDFs, website pages, or handoff pointers.

20.3 AB Mode and Gate Semantics Profile

Release writes, tag creation, site publication, archive upload, and portable package writes are material mutations.

`AB=B` alone is not enough.

They require explicit gate and human/c approval.

20.4 Registry Profile

Registry may identify agents allowed to prepare release artifacts.

Registry does not grant publish authority.

20.5 Local Checker Profile

Local checker verifies release-surface consistency.

It does not certify conformance and does not authorize publication.

20.6 Readiness Gate Profile

Readiness Gate defines claim levels.

This profile binds those claims to visible release artifacts and public/archival surfaces.

21. Conformance gates

Gate	Name	Blocking failure
`RPS-G0`	Status honesty	release claim stronger than evidence
`RPS-G1`	Canonical inventory	missing/duplicate canonical files
`RPS-G2`	Public/restricted split	restricted contamination
`RPS-G3`	Redaction	secrets/raw evidence/live targets in public
`RPS-G4`	Discoverability	required docs not visible from default surface
`RPS-G5`	Artifact order	PDFs/hashes before freeze
`RPS-G6`	Manifest integrity	manifest does not match files
`RPS-G7`	Website consistency	site/JSON stale or contradictory

Gate	Name	Blocking failure
RPS-G8	Archive consistency	Zenodo/DOI metadata mismatch
RPS-G9	Human/c gate	public/archive action without required gate
RPS-G10	Codex boundary	worker attempts autonomous publication
RPS-G11	Portable integrity	USB/package import without manifest/hash check

22. Open issues

ID	Issue	Required action
RPS-OI-001	Final repository path unknown	Decide package root path before public release.
RPS-OI-002	Public/restricted bundle layout	Decide whether restricted files live in separate directory, private repo, encrypted package, or review-only channel.
RPS-OI-003	Website metadata schema	Define final JSON fields for website publication page.
RPS-OI-004	Zenodo metadata template	Create archive metadata template for future release.
RPS-OI-005	PDF generation pipeline	Define exact PDF generation command and style after Markdown freeze.
RPS-OI-006	SHA256SUMS policy	Define whether hashes cover only public artifacts or public+restricted separately.
RPS-OI-007	Portable package policy	Decide if CGAM package is included in USB/recovery packages or only source archives.
RPS-OI-008	Local checker integration	Implement release-surface checks in local checker.
RPS-OI-009	Public summary of restricted profiles	Create redacted summaries for DEB / IRP / SCDP if public package excludes full versions.
RPS-OI-010	Schema extraction dependency	Align release-complete vs implementation-ready claims after schema extraction.

23. Closing rule

A public release is a promise about what readers and machines are allowed to treat as current.

Do not make that promise from a dirty worktree.

Do not make that promise from a side branch.

Do not make that promise with stale status files.

Do not make that promise by publishing restricted material and calling it transparency.

Final rule:

```
Make the package visible only after it is bounded.
Make it archival only after it is frozen.
Make it implementation-ready only after machines can check it.
```